



Library Management System

Spring Boot Application — Books & Authors

Course

Big Data Analytics (BDA)

GitHub

<https://github.com/himan0109/library-management-system>

Tech Stack

Spring Boot 2.7 · JPA/Hibernate · H2 · JSP · JUnit 5 · Mockito

Date

May 2026

 Table of Contents

1. Introduction & Overview
2. Entity Relationship Design
3. Project Architecture
4. Implementation Details — Entity Layer
5. Implementation Details — Repository Layer
6. Implementation Details — Service Layer
7. Implementation Details — Controller Layer
8. Implementation Details — View Layer (JSP)
9. Database Seeding (10 × 10 Sample Data)
10. Exception Handling
11. Testing — Repository & Service Unit Tests
12. Application Screenshots
13. Challenges Faced & Solutions
14. GitHub Repository

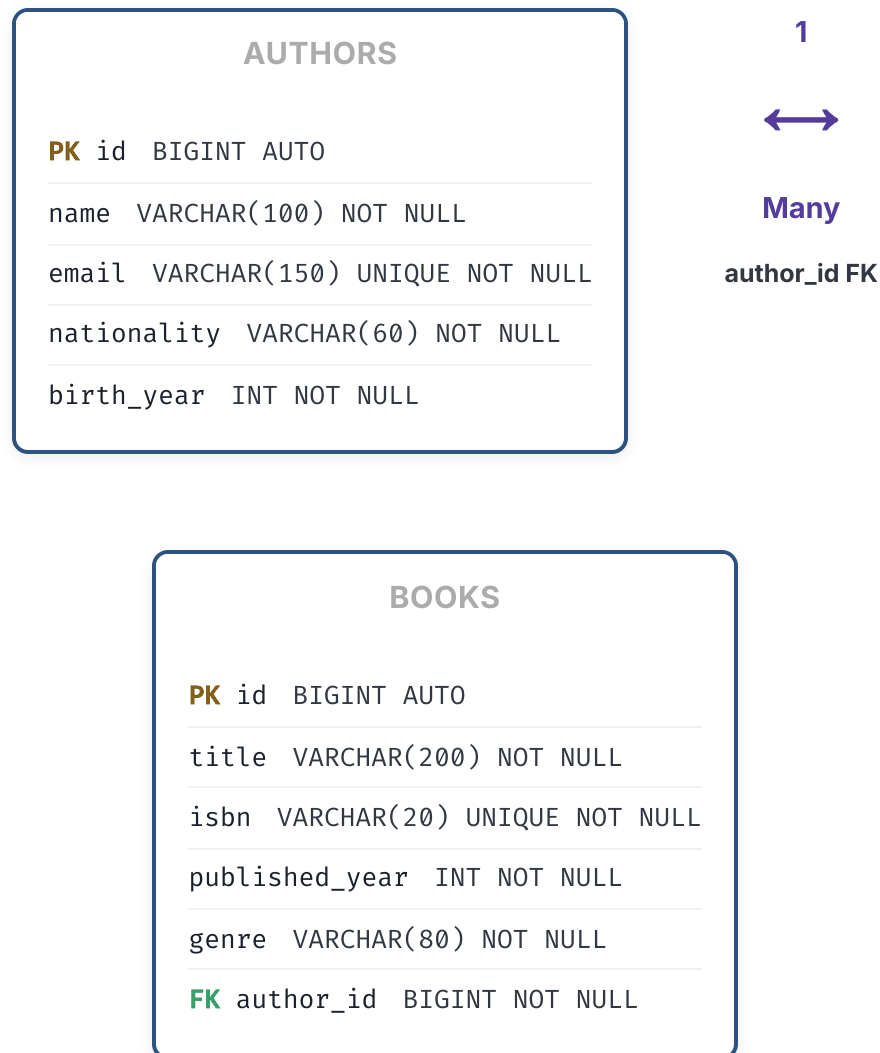
1 • Introduction & Overview

The **Library Management System** is a full-stack Spring Boot web application that manages two core entities — **Authors** and **Books** — and exposes Create, Read, and Update operations through a browser-based JSP interface.

Requirement	Implementation
Two entities with a relationship	Author (1) ↔ Book (Many) via @OneToMany / @ManyToOne
Populate database	DataInitializer seeds 10 authors + 10 books on startup
Create operation	Form-based POST with Bean Validation + integrity error handling
Read operation	List views + custom INNER JOIN JPQL query returning a DTO
Update operation	Pre-filled edit form, POST updates, redirect-after-post
Unit tests	32 tests across AuthorRepository, BookRepository, AuthorService, BookService

2 · Entity Relationship Design

The two entities share a **One-to-Many** relationship: one Author can write many Books, and each Book belongs to exactly one Author.



Cardinality: One Author → Many Books. The author_id foreign key in the BOOKS table enforces referential integrity at the database level. Cascading ALL on the Author side ensures books are removed when their author is deleted.

3 · Project Architecture

The application follows the standard Spring MVC layered architecture with strict separation of concerns:



View Layer — JSP Pages (authors/, books/, index.jsp, error.jsp)

▲ Model & View binding (Spring ModelAttribute / JSTL EL) ▼



Controller Layer — AuthorController, BookController, HomeController

▲ DTO / Entity passing ▼



Service Layer — AuthorServiceImpl, BookServiceImpl

▲ Spring Data JPA ▼



Repository Layer — AuthorRepository, BookRepository (JpaRepository)

▲ JDBC ▼



Database — H2 In-Memory (auto-created by Hibernate DDL)

Package structure

```
com.bda.library
├── LibraryApplication.java
├── config/      DataInitializer.java
├── controller/  AuthorController · BookController · HomeController
├── dto/         BookWithAuthorDTO.java
├── entity/      Author · Book
├── exception/   GlobalExceptionHandler.java
├── repository/  AuthorRepository · BookRepository
├── service/
│   ├── AuthorService (interface) · BookService (interface)
│   └── impl/ AuthorServiceImpl · BookServiceImpl
```

4 · Implementation — Entity Layer

Author.java

```
@Entity
@Table(name = "authors")
public class Author {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 100)
    @Column(nullable = false, length = 100)
    private String name;

    @Email @NotBlank
    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @NotBlank
    private String nationality;

    @Min(1000) @Max(2025)
    private int birthYear;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Book> books = new ArrayList<>();
}
```

Book.java

```
@Entity
@Table(name = "books")
public class Book {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 200) private String title;

    @NotBlank @Pattern(regexp = "^[0-9\\-]{10,17}$")
    @Column(unique = true) private String isbn;

    @Min(1000) @Max(2025) private int publishedYear;
```

```
@NotBlank private String genre;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "author_id", nullable = false)
private Author author;
}
```

Key JPA annotations used:

Annotation	Purpose
@Entity	Marks the class as a JPA-managed entity mapped to a database table
@Table(name= ...)	Explicitly names the database table
@Id @GeneratedValue	Primary key with auto-increment strategy
@Column(unique=true)	Adds a UNIQUE constraint (email, ISBN)
@OneToMany	Author owns the one side of the relationship
@ManyToOne @JoinColumn	Book holds the FK column author_id
@NotBlank @Email @Min @Max	Bean Validation constraints checked before persistence

5 · Implementation — Repository Layer

Both repositories extend `JpaRepository<T, ID>`, which automatically provides `save()`, `findById()`, `findAll()`, `deleteById()`, and more.

Custom query methods

```
// AuthorRepository
Optional<Author> findByEmail(String email);
List<Author> findByNationality(String nationality);
boolean existsByEmail(String email);

@Query("SELECT a FROM Author a WHERE LOWER(a.name) LIKE LOWER(CONCAT('%',:name,'%'))")
List<Author> searchByName(String name);
```

Custom INNER JOIN query (BookRepository)

```
@Query("""
    SELECT new com.bda.library.dto.BookWithAuthorDTO(
        b.id, b.title, b.isbn, b.publishedYear, b.genre,
        a.id, a.name, a.nationality)
    FROM Book b INNER JOIN b.author a
    ORDER BY a.name, b.title
    """)
List<BookWithAuthorDTO> findAllBooksWithAuthorDetails();

// Fetch-join avoids N+1 problem when loading the books list
@Query("SELECT b FROM Book b INNER JOIN FETCH b.author a ORDER BY b.title")
List<Book> findAllWithAuthor();
```

The JPQL INNER JOIN query projects results into a `BookWithAuthorDTO` object (using the new constructor expression), satisfying the requirement for a custom join query between both entities. The SQL equivalent is: `SELECT b.*, a.name, a.nationality FROM books b INNER JOIN authors a ON b.author_id = a.id`

6 · Implementation — Service Layer

Service classes implement their respective interfaces and are annotated with `@Service` `@Transactional`. Read-only operations use `@Transactional(readOnly = true)` for performance.

AuthorServiceImpl — update method

```
@Override
public Author update(Long id, Author updated) {
    Author existing = authorRepository.findById(id)
        .orElseThrow(() -> new EntityNotFoundException("Author not found: " + id));
    existing.setName(updated.getName());
    existing.setEmail(updated.getEmail());
    existing.setNationality(updated.getNationality());
    existing.setBirthYear(updated.getBirthYear());
    return authorRepository.save(existing);
}
```



BookServiceImpl — save method (author resolution)

```
@Override
public Book save(Book book) {
    // Re-fetch the full Author entity by the ID submitted from the form
    Author author = authorRepository.findById(book.getAuthor().getId())
        .orElseThrow(() -> new EntityNotFoundException("Author not found"));
    book.setAuthor(author);
    return bookRepository.save(book);
}
```

7 · Implementation — Controller Layer

URL Mapping Summary

Method	URL	Action
GET	/authors	List all authors (Read)
GET	/authors/new	Show create form (Create)
POST	/authors	Save new author (Create)
GET	/authors/{id}/edit	Show edit form (Update)
POST	/authors/{id}/update	Save updated author (Update)
GET	/books	List all books (Read)
GET	/books/joined	INNER JOIN view (Read)
GET	/books/new	Show create form (Create)
POST	/books	Save new book (Create)
GET	/books/{id}/edit	Show edit form (Update)
POST	/books/{id}/update	Save updated book (Update)

Create handler with validation & integrity error handling

```
@PostMapping
public String createAuthor(@Valid @ModelAttribute("author") Author author,
                           BindingResult bindingResult, Model model,
                           RedirectAttributes redirectAttrs) {
    if (bindingResult.hasErrors()) {           // Bean Validation failed
        model.addAttribute("formAction", "/authors");
        return "authors/form";                // redisplay with error messages
    }
    try {
        authorService.save(author);
        redirectAttrs.addFlashAttribute("successMessage", "Author created.");
    } catch (DataIntegrityViolationException e) {
```

```
        model.addAttribute("errorMessage", "Email already exists.");  
        return "authors/form";  
    }  
    return "redirect:/authors";           // Post-Redirect-Get pattern  
}
```

The same pattern is applied for Books, with duplicate ISBN detected via `DataIntegrityViolationException`.

8 · Implementation — View Layer (JSP)

JSP pages created

File	Purpose
index.jsp	Home dashboard with navigation cards
authors/list.jsp	Tabular list of all authors with Edit button
authors/form.jsp	Reused for both Create and Update (formAction attribute switches the POST target)
books/list.jsp	Tabular list of all books with genre badges and author name
books/form.jsp	Book form with author <select> dropdown; reused for Create and Update
books/joined.jsp	Displays result of the INNER JOIN JPQL query with the SQL shown
error.jsp	Global error page rendered by GlobalExceptionHandler

Data binding example (books/form.jsp)

```

<!-- Spring form tag binds to 'book' model attribute -->
<form:form action="${formAction}" method="post" modelAttribute="book">
  <form:input path="title" placeholder="e.g. 1984" />
  <form:errors path="title" cssClass="error" />

  <form:select path="author.id">
    <form:option value="" label="-- Select an Author --" />
    <c:forEach var="a" items="${authors}">
      <form:option value="${a.id}" label="${a.name}" />
    </c:forEach>
  </form:select>
</form:form>

```

All pages use embedded CSS with a consistent navy/blue colour scheme, responsive flex layout, hover effects on table rows, and badge labels for genres and nationalities.

9 · Database Seeding — 10 × 10 Sample Data

DataInitializer implements ApplicationRunner and runs once on startup, inserting 10 authors and 10 books if the tables are empty.

Authors (10 rows)

#	Name	Email	Nationality	Birth Year
1	George Orwell	gorwell@mail.com	British	1903
2	J.K. Rowling	jkrowling@mail.com	British	1965
3	Fyodor Dostoevsky	dostoevsky@mail.com	Russian	1821
4	Gabriel García Márquez	gmarquez@mail.com	Colombian	1927
5	Toni Morrison	tmorrison@mail.com	American	1931
6	Haruki Murakami	murakami@mail.com	Japanese	1949
7	Leo Tolstoy	tolstoy@mail.com	Russian	1828
8	Jane Austen	jausten@mail.com	British	1775
9	Mark Twain	mtwain@mail.com	American	1835
10	Franz Kafka	kafka@mail.com	Czech	1883

Books (10 rows)

#	Title	ISBN	Year	Genre	Author
1	1984	978-0451524935	1949	Dystopian Fiction	George Orwell
2	Harry Potter and the Philosopher's Stone	978-0747532743	1997	Fantasy	J.K. Rowling
3	Crime and Punishment	978-0486454115	1866	Psychological Fiction	Fyodor Dostoevsky

4	One Hundred Years of Solitude	978-0060883287	1967	Magical Realism	García Márquez
5	Beloved	978-1400033416	1987	Historical Fiction	Toni Morrison
6	Norwegian Wood	978-0375704024	1987	Literary Fiction	Haruki Murakami
7	War and Peace	978-1400079988	1869	Historical Fiction	Leo Tolstoy
8	Pride and Prejudice	978-0141439518	1813	Romance	Jane Austen
9	The Adventures of Tom Sawyer	978-0486400778	1876	Adventure	Mark Twain
10	The Metamorphosis	978-0486290300	1915	Absurdist Fiction	Franz Kafka

10 · Exception Handling

A `@ControllerAdvice` class — `GlobalExceptionHandler` — catches exceptions thrown anywhere in the controller layer and renders a friendly error page.

Exception	Cause	User message
<code>DataIntegrityViolationException</code>	Duplicate email or ISBN in the database	Inline error on the form (not redirected to error page)
<code>EntityNotFoundException</code>	Invalid ID in URL (e.g. <code>/authors/999/edit</code>)	Error page: "Record Not Found"
Exception (generic)	Any unexpected runtime error	Error page: "Unexpected Error"

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(DataIntegrityViolationException.class)
    public String handleDataIntegrity(DataIntegrityViolationException ex, Model model) {
        model.addAttribute("errorMessage", "Constraint violation – check for duplicate");
        return "error";
    }
}
```

11 • Testing

32 unit tests are organised into four test classes:

Test Class	Type	Tests	Framework
AuthorRepositoryTest	Integration (@DataJpaTest)	8	JUnit 5 + H2
BookRepositoryTest	Integration (@DataJpaTest)	7	JUnit 5 + H2
AuthorServiceTest	Unit (mocked)	8	JUnit 5 + Mockito
BookServiceTest	Unit (mocked)	9	JUnit 5 + Mockito

Repository test example — custom INNER JOIN query

```
@Test
@DisplayName("findAllBooksWithAuthorDetails() inner join returns DTO rows")
void findAllBooksWithAuthorDetails_returnsJoinResult() {
    List<BookWithAuthorDTO> dtos = bookRepository.findAllBooksWithAuthorDetails();
    assertThat(dtos).isNotEmpty();
    assertThat(dtos.get(0).getTitle()).isEqualTo("1984");
    assertThat(dtos.get(0).getAuthorName()).isEqualTo("George Orwell");
}
```

Service test example — update throws when author not found

```
@Test
void update_throwsWhenNotFound() {
    when(authorRepository.findById(99L)).thenReturn(Optional.empty());
    assertThatThrownBy(() -> authorService.update(99L, author))
        .isInstanceOf(EntityNotFoundException.class)
        .hasMessageContaining("99");
}
```

All 32 tests use AssertJ fluent assertions for readable failure messages. Repository tests use the @DataJpaTest slice which boots only the JPA layer against a real H2 database — ensuring queries are validated against an actual SQL engine.

12 · Application Screenshots

Run the application with `mvn spring-boot:run` and navigate to `http://localhost:8080` to capture screenshots. Replace the placeholders below before submitting.

Home Page

[Screenshot: `http://localhost:8080/`]

Dashboard with Books, Authors, and Join View cards.

Authors List (Read)

[Screenshot: `http://localhost:8080/authors`]

Table showing all 10 seeded authors with Edit buttons.

Add Author Form (Create)

[Screenshot: `http://localhost:8080/authors/new`]

Create form with field validation errors visible.

Edit Author Form (Update)

[Screenshot: `http://localhost:8080/authors/1/edit`]

Pre-filled edit form for George Orwell.

Books List (Read)

[Screenshot: <http://localhost:8080/books>]

Table of 10 books with genre badges and author names.

Add Book Form (Create)

[Screenshot: <http://localhost:8080/books/new>]

Form with author dropdown populated from database.

Inner Join View (Custom Query)

[Screenshot: <http://localhost:8080/books/joined>]

INNER JOIN result with SQL query displayed, all 10 books paired with author info.

Duplicate ISBN Error (Exception Handling)

[Screenshot: submit a book form with an existing ISBN]

Inline error message: "A book with this ISBN already exists."

Unit Tests Passing

[Screenshot: terminal showing mvn test output — BUILD SUCCESS, 32

tests run, 0 failures]

13 · Challenges Faced & Solutions

Challenge	Solution
N+1 Query Problem — loading books list triggered a separate SQL query per book to fetch the author.	Added a <code>INNER JOIN FETCH</code> JPQL query (<code>findAllWithAuthor()</code>) in <code>BookRepository</code> so the author is loaded eagerly in one SQL statement.
Author binding in Book form — the form submits only <code>author.id</code> , but JPA needs a managed <code>Author</code> entity.	The <code>BookServiceImpl.save()</code> method re-fetches the full <code>Author</code> entity from the repository using the submitted ID before calling <code>bookRepository.save()</code> .
Duplicate constraint errors — H2 throws a low-level <code>DataIntegrityViolationException</code> when a duplicate email or ISBN is inserted, without a user-friendly message.	Caught in the controller's try-catch block and in <code>GlobalExceptionHandler</code> ; a human-readable message is added to the model and the form is re-rendered.
Reusing the same JSP form for Create and Update — the form action URL differs between the two operations.	Controllers pass a <code>formAction</code> string attribute to the model. The JSP's <code><form:form action="\${formAction}"></code> uses this dynamically, so one template handles both cases.
JPQL DTO projection — mapping a multi-table query result to a plain Java object without involving a full entity.	Used the JPQL new <code>com.bda.library.dto.BookWithAuthorDTO(...)</code> constructor expression, passing selected fields directly into the DTO constructor.

14 · GitHub Repository

```
https://github.com/himan0109/library-management-system
```

The repository contains the complete Maven project. To run locally:

```
git clone https://github.com/himan0109/library-management-system.git
cd library-management-system
mvn spring-boot:run
# Open http://localhost:8080 in your browser
```

To run all 32 unit tests:

```
mvn test
```